

Atividade Avaliativa

Design Patterns em Python

Sistema de pedidos

Isaías Gouvêa Gonçalves

Mateus Mourão

Estrutura da solução

A solução foi dividida em módulos de acordo com a responsabilidade de cada componente. O diretório `src/` contém o código da aplicação: `app_config.py` concentra a configuração compartilhada; `order.py` reúne o pedido, os produtos e o Builder; `payment.py` contém a hierarquia de pagamentos e o Factory Method; `channels.py` define as famílias WEB e MOBILE; `kiosk_channel.py` acrescenta a família KIOSK; `channel_registry.py` mantém o registro de fábricas; `order_service.py` coordena o fluxo; e `event_logger.py` registra os eventos.

O diretório `tests/` contém os testes separados por componente e os testes de integração. O arquivo `main.py` funciona como raiz de composição e exemplo executável: configura as fábricas, constrói um pedido e executa o fluxo completo nos canais WEB e KIOSK. Essa organização mantém as responsabilidades separadas e permite alterar ou estender um componente sem concentrar toda a aplicação em um único módulo.

1 - Config (Singleton)

1. Explique por que utilizar `__new__` não impede, por si só, novas execuções de `__init__`.

Sempre que `AppConfig()` é invocado, `__new__` determina qual instância será retornada. Se o objeto retornado for uma instância da classe, o protocolo normal de construção ainda chama `__init__`, mesmo que `__new__` tenha devolvido um objeto já existente. Portanto, usar apenas `__new__` para controlar a identidade não impede que um eventual `__init__` seja executado novamente e restaure atributos. Em nossa implementação, os valores iniciais são definidos somente dentro da condição que cria a primeira instância.

2. Explique como um módulo Python poderia ser utilizado para compartilhar uma configuração sem implementar uma segunda versão do Singleton.

Uma alternativa seria criar um módulo `config.py` que armazenasse diretamente os valores compartilhados:

```
ENVIRONMENT = "production"
CURRENCY = "BRL"
DEBUG = False
```

Quando as configurações fossem utilizadas, bastaria importar o módulo:

```
import config

print(f"Moeda utilizada: {config.CURRENCY}") # BRL
```

Como os módulos são carregados uma vez e mantidos no cache de importações de Python, todos os arquivos que importassem `config` acessariam o mesmo objeto de módulo e, conseqüentemente, os mesmos valores. Assim, o próprio mecanismo de módulos forneceria o compartilhamento, sem implementar outra classe Singleton.

3. Identifique uma possível consequência de possuir um objeto de configuração global compartilhado.

Uma possível consequência é o acoplamento entre componentes independentes que acessam o mesmo estado global. Qualquer componente pode alterar uma configuração e produzir efeitos inesperados no restante da aplicação. Os testes também podem interferir uns nos outros; por isso, a suíte redefine `AppConfig._instance` no `setUp()` para começar cada teste com uma configuração conhecida.

2 - Pedido e Builder

1. Identifique quais componentes da sua implementação correspondem ao Builder e ao objeto construído.

A classe `OrderBuilder` corresponde ao Builder. Ela mantém temporariamente os dados usados na construção, como cliente, produtos, endereço, cupom, forma de pagamento e observação. Seus métodos `set_customer()`, `add_product()`, `set_address()`, `set_coupon()`, `set_payment_method()` e `set_obs()` representam as etapas de configuração e retornam o próprio Builder para permitir encadeamento. O método `build()` encerra o processo de construção. Ele verifica se o cliente obrigatório foi informado e cria uma instância de `Order`. A classe `Order` corresponde ao objeto construído. Ela representa o pedido final e contém seus dados e comportamentos, como o método `total()`. A classe `Product` representa os produtos que compõem o pedido, mas não é o produto final do padrão Builder neste caso.

2. Explique por que seria possível construir o pedido diretamente pelo construtor de `Order` e qual seria a diferença em relação à solução adotada.

Seria possível construir o pedido diretamente porque o construtor de `Order` já recebe todas as informações necessárias:

```
order = Order(
    customer="Luis Bueno",
    products=[keyboard, mouse],
    address="Rua Exemplo, 42",
    coupon="DESCONTO10",
)
```

Em Python, os argumentos nomeados tornam essa construção relativamente clara, especialmente quando o objeto possui poucos campos e regras simples. Na solução adotada, `OrderBuilder` permite que o pedido seja configurado em etapas, usando uma interface encadeada. O Builder mantém o estado intermediário da construção, permite adicionar produtos individualmente e centraliza a validação de que um cliente foi informado antes de criar o pedido. Outra diferença é que o Builder copia a lista de produtos ao construir o `Order`. Dessa forma, reutilizar o mesmo Builder para criar outro pedido não modifica retroativamente um pedido já construído. A construção direta seria mais curta, mas colocaria sobre o código cliente a responsabilidade de preparar todos os argumentos e poderia contornar a validação realizada em `build()`. O Builder se torna mais vantajoso caso o processo de criação ganhe novas etapas ou regras.

3 - Pagamento e Factory Method

1. Identifique os papéis de Creator, Concrete Creator, Product e Concrete Product na implementação desenvolvida.

Na implementação, `PaymentProcessor` desempenha o papel de Creator, pois declara o Factory Method `create_payment()` e define em `process_order()` o fluxo comum de processamento.

`PixProcessor`, `CreditCardProcessor` e `BoletoProcessor` são os Concrete Creators, pois implementam `create_payment()` e decidem qual objeto de pagamento será criado.

A classe abstrata `Payment` representa o Product, estabelecendo o contrato `pay(amount)`.

Por fim, `PixPayment`, `CreditCardPayment` e `BoletoPayment` são os Concrete Products, pois implementam esse contrato para cada mecanismo de pagamento. Dessa forma, o fluxo definido em `PaymentProcessor` depende da abstração `Payment`, enquanto a escolha da implementação concreta fica a cargo de cada processor especializado.

2. Explique por que uma função contendo simplesmente uma sequência de if/elif escolhendo classes concretas não é, por si só, suficiente para caracterizar o padrão Factory Method.

Uma função com uma sequência de `if/elif` apenas centraliza a escolha e a instanciação das classes concretas, caracterizando uma Simple Factory, e não necessariamente o padrão Factory Method.

No Factory Method, a criação do produto é definida por um método que pode ser sobrescrito por subclasses. Em nossa implementação, `PaymentProcessor` mantém o fluxo comum em `process_order()`, enquanto `PixProcessor`, `CreditCardProcessor` e `BoletoProcessor` implementam `create_payment()` de forma polimórfica. Assim, cada subclasse decide qual `Payment` criar sem que o fluxo geral precise conhecer ou selecionar diretamente as classes concretas.

Em uma função baseada em `if/elif`, adicionar uma forma de pagamento normalmente exigiria alterar essa própria função e incluir outra condição.

3. Considere que uma nova forma de pagamento seja adicionada posteriormente. Explique quais partes da implementação precisariam ser alteradas.

Para adicionar uma nova forma de pagamento, bastaria criar uma nova implementação concreta de `Payment` e um novo `PaymentProcessor` que implemente `create_payment()` retornando esse pagamento.

Também seria necessário disponibilizar o novo processor na inicialização da aplicação. O fluxo comum de `PaymentProcessor.process_order()` e as classes de pagamento existentes não precisariam ser modificados.

4 - Famílias por canal e Abstract Factory

1. Explique por que checkout e notificação podem ser considerados uma família de produtos.

Checkout e Notification são duas variações que sempre aparecem juntas e precisam ser coerentes entre si: um pedido feito pelo canal WEB deve receber um WebCheckout e uma WebNotification; um pedido pelo MOBILE deve receber MobileCheckout e MobileNotification. Não faz sentido misturar, por exemplo, um checkout WEB com uma notificação MOBILE. Como esses objetos são criados aos pares, de forma consistente, e compartilham a mesma "variante" (o canal), eles formam uma família de produtos relacionados -- exatamente o que a Abstract Factory (ChannelFactory) foi feita para produzir.

2. Explique qual problema a Abstract Factory resolve nessa situação.

Sem a Abstract Factory, o código que monta o fluxo do pedido precisaria de lógica condicional (`if channel == "WEB": ... elif channel == "MOBILE": ...`) espalhada em vários pontos para escolher tanto o checkout quanto a notificação certos, correndo o risco de, em algum lugar, combinar classes de canais diferentes por engano. A ChannelFactory resolve isso concentrando em um único objeto (WebFactory, MobileFactory, ...) a responsabilidade de criar toda a família compatível de um canal. Quem usa a fábrica só chama `create_checkout()` e `create_notification()` sem saber qual canal está por trás, e a garantia de que os dois objetos pertencem ao mesmo canal fica embutida na própria fábrica.

3. Explique por que o pagamento não deve fazer parte da fábrica responsável pelo canal.

Canal de venda e forma de pagamento são escolhas independentes (WEB pode usar PIX, MOBILE pode usar cartão, e vice-versa). Se ChannelFactory também criasse o Payment, canal e forma de pagamento ficariam acoplados -- não seria possível combinar livremente um canal com qualquer forma de pagamento, e adicionar uma nova forma de pagamento exigiria mexer em todas as fábricas de canal. Ao manter o pagamento fora da ChannelFactory e resolvido por um PaymentProcessor separado (Factory Method da questão 3), as duas dimensões variam de forma independente, cada uma com sua própria responsabilidade (SRP).

5 - Seleção de fábrica e canal KIOSK

1. Liste os arquivos criados ou alterados para adicionar KIOSK.

Criado: `src/kiosk_channel.py` (com `KioskCheckout`, `KioskNotification` e `KioskFactory`, implementando as mesmas abstrações de `src/channels.py`).

Alterado: `main.py`, só na inicialização (`bootstrap()`), acrescentando o `import` de `KioskFactory` e a chamada `register_factory("KIOSK", KioskFactory())`.

Não alterados: `src/channel_registry.py` (o `get_channel_factory` e o mecanismo de registro continuam exatamente os mesmos), `src/channels.py` e `src/order_service.py` (o código que consome `Checkout/Notification` através das abstrações não precisou saber que KIOSK existe).

2. Explique por que as alterações realizadas são ou não compatíveis com o princípio OCP.

São compatíveis com o OCP. O módulo de registro (`channel_registry.py`) está fechado para modificação: `get_channel_factory` não teve nenhuma linha alterada e continua funcionando para qualquer canal que já esteja no dicionário `_factories`, seja `WEB`, `MOBILE` ou `KIOSK`. Ao mesmo tempo, o sistema está aberto para extensão: para suportar um canal novo bastou criar um arquivo novo que implementa as interfaces já existentes (`ChannelFactory`, `Checkout`, `Notification`) e registrá-lo. A única mudança em código já existente foi em `main.py`, que é a raiz de composição da aplicação (o lugar cuja função é justamente "ligar" os componentes); nenhuma classe que participa da lógica de negócio (`OrderService`, `channel_registry.py`, `channels.py`) precisou ser reaberta ou reescrita.

6 - Responsabilidades e integração

As respostas desta seção devem considerar a implementação efetivamente entregue pela dupla.

A primeira versão de `OrderService` misturava cinco responsabilidades (`create_order`, `calculate_total`, `create_payment`, `send_notification`, `save_log`). Elas foram redistribuídas assim:

- `create_order` -> `OrderBuilder` (constrói o pedido);
- `calculate_total` -> `Order.total()` (o próprio pedido sabe somar seus itens);
- `create_payment` -> `PaymentProcessor.create_payment()` de cada processador concreto (Factory Method, questão 3);
- `send_notification` -> `Notification.send(order)`, obtida de uma `ChannelFactory` (Abstract Factory, questão 4);
- `save_log` -> `EventLogger`, um componente próprio dedicado só a registrar o que aconteceu no fluxo.

O `OrderService` que sobrou só coordena: recebe por injeção um `PaymentProcessor` e uma `ChannelFactory` já prontos e chama, em sequência, `checkout.show(order)`, `payment_processor.process_order(order)`, `notification.send(order)` e (se houver) `logger.log(...)`. Ele não constrói o pedido, não decide qual classe concreta de pagamento ou notificação usar, e não guarda a configuração global -- essas decisões são tomadas antes, na inicialização da aplicação (`main.py`), que registra as fábricas de canal e escolhe o `PaymentProcessor` a usar.

1. Qual é a responsabilidade principal de cada componente criado?

- `AppConfig`: manter a configuração compartilhada da aplicação (Singleton).
- `Product`: representar um item com nome e preço.
- `Order / OrderBuilder`: representar e construir um pedido válido.
- `Payment / PixPayment / CreditCardPayment / Boletopayment / PayPalPayment`: definir e executar uma forma específica de pagamento.
- `PaymentProcessor` (e subclasses `PixProcessor`, `CreditCardProcessor`, `BoletoProcessor`, `PayPalProcessor`): definir o fluxo comum de processamento de pagamento e delegar a criação do `Payment` concreto ao Factory Method.
- `Checkout / Notification`: apresentar o checkout e enviar a notificação de um canal específico.
- `ChannelFactory` (`WebFactory`, `MobileFactory`, `KioskFactory`): criar a família coerente de `Checkout` e `Notification` de um canal (Abstract Factory).
- `channel_registry` (`register_factory / get_channel_factory`): saber, em tempo de execução, qual `ChannelFactory` corresponde a um canal, sem que quem consulta precise conhecer as classes concretas.
- `EventLogger`: registrar os eventos ocorridos durante o processamento de um pedido.
- `OrderService`: coordenar o fluxo completo chamando os colaboradores acima, na ordem certa, sem tomar decisões que pertencem a eles.
- `main.py`: raiz de composição -- registra as fábricas de canal, escolhe o `PaymentProcessor` e dispara o fluxo.

2. Escolha três componentes diferentes e indique uma mudança que deveria ficar restrita a cada um deles.

- `WebCheckout/WebNotification`: mudar o texto ou o layout apresentado ao cliente no canal WEB (ex.: incluir um novo campo no checkout) deveria alterar só essas duas classes, sem afetar `MOBILE`, `KIOSK` ou o `OrderService`.
- `channel_registry.py`: mudar a forma como as fábricas são armazenadas internamente (por exemplo, trocar o dicionário por outra estrutura, ou passar a validar duplicidade de registro)

deveria ficar restrito a este arquivo -- quem chama `get_channel_factory(channel)` não percebe a diferença.

- `EventLogger`: mudar o destino dos logs (por exemplo, gravar em arquivo em vez de manter em memória, ou mudar o formato do timestamp) deveria ficar restrito a essa classe; `OrderService` continua só chamando `logger.log(mensagem)`.

3. Identifique uma decisão de projeto da solução que poderia ser diferente. Explique qual seria a alternativa e qual seria a consequência dessa mudança.

Optamos por registrar as fábricas de canal (`register_factory`) explicitamente em `main.py`, na inicialização da aplicação. Uma alternativa seria fazer cada módulo de canal se auto-registrar como efeito colateral da importação (por exemplo, `src/channels.py` chamar `register_factory("WEB", WebFactory())` no fim do próprio arquivo, e `src/kiosk_channel.py` fazer o mesmo para KIOSK). Nesse caso, adicionar KIOSK exigiria zero alterações em `main.py` -- bastaria importar o novo módulo em algum ponto de inicialização. A consequência seria um acoplamento mais implícito: o registro de um canal passaria a depender de "alguém importar aquele módulo em algum lugar", o que é mais fácil de esquecer e mais difícil de rastrear do que ler as poucas linhas de `bootstrap()` em `main.py`. Preferimos a explicitação em `main.py` por deixar visível, em um único lugar, quais canais a aplicação suporta.

7 - Testes adicionais

Para cada teste, informe o comportamento verificado, o resultado esperado e por que esse comportamento é importante para a solução. Os comportamentos não devem ser exatamente os exemplos apresentados no enunciado.

Teste adicional de Singleton

Arquivo `tests/test_app_config.py`, método `test_valores_iniciais`.

Comportamento verificado

O teste verifica se a primeira obtenção de `AppConfig` cria a configuração com os valores padrão definidos para a aplicação: ambiente "production", moeda "BRL" e modo de depuração desativado.

Resultado esperado

Espera-se que `config.environment` seja "production", `config.currency` seja "BRL" e `config.debug` seja `False`.

Por que esse comportamento é importante

Esse comportamento garante que o único objeto compartilhado comece em um estado válido e previsível. Como todas as partes da aplicação utilizam a mesma instância, um valor inicial incorreto seria propagado para todos os componentes. O teste é adicional porque os exemplos obrigatórios se concentram na identidade e no compartilhamento da instância, enquanto este verifica seu estado inicial.

Teste adicional de Builder

Arquivo `tests/test_order.py`, método `test_pedido_construido_nao_muda_ao_reutilizar_builder`.

Comportamento verificado

O teste verifica se um pedido já construído permanece inalterado quando o mesmo `OrderBuilder` é reutilizado e recebe um novo produto antes de construir outro pedido.

Resultado esperado

Espera-se que o primeiro pedido continue com apenas um produto e que o segundo possua dois. O produto acrescentado depois da primeira chamada a `build()` não deve aparecer retroativamente no primeiro pedido.

Por que esse comportamento é importante

Esse comportamento é importante porque o Builder mantém uma lista mutável durante a construção. A cópia realizada por `build()` impede que pedidos diferentes compartilhem acidentalmente a mesma lista e evita alterações retroativas em objetos já entregues ao restante do sistema.

Teste adicional de criação por Factory

Arquivo `tests/test_channels.py`, método `test_fabrica_cria_uma_nova_instancia_a_cada_chamada`.

Comportamento verificado

Que duas chamadas sucessivas a `WebFactory.create_checkout()` devolvem duas instâncias distintas de `WebCheckout`, e não o mesmo objeto reaproveitado.

Resultado esperado

`checkout_1 is not checkout_2` -- os dois objetos criados são instâncias diferentes.

Por que esse comportamento é importante

O enunciado só pede para verificar que a fábrica cria o tipo certo de objeto para cada canal. Este teste cobre um comportamento diferente: confirma a decisão de projeto de produzir um novo checkout a cada chamada, em vez de reutilizar uma instância. Isso importa porque pedidos processados em paralelo usam a mesma `ChannelFactory`; se ela devolvesse sempre o mesmo `Checkout`, uma futura inclusão de estado nessa classe poderia fazer pedidos diferentes compartilharem dados acidentalmente.

8 - Nova forma de pagamento

1. Quais arquivos foram criados ou modificados?

Não foi necessário criar um novo arquivo. O arquivo `src/payment.py` foi modificado para incluir `PayPalPayment`, a nova implementação concreta de `Payment`, e `PayPalProcessor`, o novo `Concrete Creator` responsável por instanciá-la.

O arquivo `tests/test_payment.py` também foi modificado para acrescentar um teste que constrói um pedido com a forma de pagamento `PayPal` e verifica se `PayPalPayment.pay()` é chamado com o valor total correto.

2. O fluxo principal de processamento precisou ser alterado?

Não. O fluxo principal de processamento não precisou ser alterado. `PayPalProcessor` herda de `PaymentProcessor` o método `process_order()`, que já cria um pagamento por meio de `create_payment()` e chama `pay(order.total())`. Para suportar o `PayPal`, bastou implementar `create_payment()` na nova subclasse, retornando uma instância de `PayPalPayment`.

3. Quais classes existentes precisaram ser modificadas?

Nenhuma classe existente precisou ser modificada. Foram apenas adicionadas as classes `PayPalPayment` e `PayPalProcessor`. As abstrações `Payment` e `PaymentProcessor`, o método `process_order()` e as demais implementações concretas permaneceram inalterados.

4. Explique como o Factory Method contribuiu para essa extensão.

O `Factory Method` permitiu adicionar o `PayPal` sem alterar o fluxo geral de processamento.

`PaymentProcessor.process_order()` depende apenas do método `create_payment()` e da abstração `Payment`, cujo contrato define `pay(amount)`. O novo `PayPalProcessor` implementa `create_payment()` retornando um `PayPalPayment`, que respeita esse mesmo contrato.

Dessa forma, o fluxo pode utilizar a nova forma de pagamento sem conhecer sua classe concreta, enquanto a decisão sobre qual objeto criar permanece encapsulada no `processor` correspondente.

5. Compare essa alteração com a inclusão do canal KIOSK. Quais são as semelhanças e diferenças arquiteturais entre as duas extensões?

As duas extensões são semelhantes porque acrescentam novas classes concretas que respeitam abstrações já existentes, sem modificar o fluxo principal da aplicação.

Isso mantém o sistema aberto para extensão e evita alterações em componentes que já funcionavam. A diferença é que o `PayPal` estende o `Factory Method` de pagamento: foram adicionados um novo produto, `PayPalPayment`, e um novo `creator`, `PayPalProcessor`, responsável por criar um único tipo de objeto. Já o canal `KIOSK` estende a `Abstract Factory`: foi necessário criar uma nova família composta por `KioskCheckout` e `KioskNotification`, além de `KioskFactory`, que produz os dois objetos de forma coerente.

A fábrica do `KIOSK` também precisou ser adicionada ao registro de canais durante a inicialização da aplicação, enquanto o `processor` do `PayPal` é escolhido diretamente como colaborador do `OrderService`.